

G09/G16 MATLAB Toolbox

A parsing and visualisation toolbox for Gaussian 09/16 output files

Sebastiano Trusso

CNR – Istituto per i Processi Chimico-Fisici (IPCF), Messina, Italy

Version 1.0 — 2026

Contents

1	Introduction	2
2	File I/O utilities	3
2.1	G09_read_lines	3
3	Core data extraction	4
3.1	G09_route / G16_route	4
3.2	G09_structure / G16_structure	4
3.3	G09_energy / G16_energy	5
3.4	G09_charge_mult / G16_charge_mult	6
3.5	G09_convergence / G16_convergence	6
3.6	G09_dipole_polar / G16_dipole_polar	7
3.7	G09_nmodes / G16_nmodes	7
3.8	G09_spectra / G16_spectra	8
4	Atomic charges	10
4.1	G09_charges / G16_charges	10
4.2	G09_charges_fchk (G09 only)	11
5	Formatted checkpoint (.fchk) support	12
5.1	G09_fchk_read (G09 only)	12
6	Molecule and normal-mode visualisation	13
6.1	G09_draw_molecule / G16_draw_molecule	13
6.2	G09_draw_mode / G16_draw_mode	13
6.3	G09_modeViewer / G16_modeViewer	14
7	Orbital energy analysis	15
7.1	G09_orbital_energies / G16_orbital_energies	15
7.2	G09_draw_orbital / G16_draw_orbital	16
8	Response properties (G16 only)	17
8.1	G16_hyerpolar	17
8.2	G16_tddft	17
9	Structural analysis utilities	19
9.1	G09_get_bond_length / G16_get_bond_length	19

9.2	G09_gaussian_version / G16_gaussian_version	19
9.3	G09_restart (G09 only)	20
9.4	G09_list / G16_list	20
10	One-call data extraction	21
10.1	G09_read_all / G16_read_all	21
11	Python package: G16parser	22
11.1	Function reference	22
11.2	Selected signatures	23
11.3	Notes on the port	24
12	Publication and licensing	25
12.1	Declaration of Generative AI and AI-assisted technologies in the writing process	25
12.2	Note on the Use of AI-Assisted Development Tools	25
12.3	Licence	25

1 Introduction

This manual documents the **G09/G16 Toolbox**, a set of MATLAB functions for parsing, analysing, and visualising Gaussian 09 and Gaussian 16 output files (`.out/.log`) and formatted checkpoint files (`.fchk`). The toolbox covers:

- extraction of molecular geometry, energies, thermochemistry, dipole moment, polarisability, and hyperpolarisability;
- vibrational analysis: normal modes, IR/Raman spectra, and interactive mode visualisation;
- molecular-orbital energy diagrams and TD-DFT excited-state analysis;
- 3D molecular structure rendering (CPK ball-and-stick models) with atomic charge and dipole overlays;
- utility functions for bond-length tables, Gaussian version detection, restart-file generation, and toolbox self-documentation.

Function names follow the `G09_xxx/G16_xxx` convention, indicating which Gaussian version's output format the function targets. Where the two versions behave identically, this manual documents them in a single combined subsection (`G09_xxx / G16_xxx`); where behaviour differs, the differences are called out explicitly.

Every parsing function that reads a `.out/.log` file accepts an optional `'Lines'` Name-Value parameter: a cell array of pre-read file lines. This lets `G09_read_all/G16_read_all` read the file from disk once and reuse the same lines across every sub-parser, instead of each function re-reading and re-splitting the file independently. Passing `'Lines'` is entirely optional — omitting it falls back to reading the file normally.

2 File I/O utilities

2.1 G09_read_lines

```
lines = G09_read_lines(filename)
```

Low-level reader used internally by every other **G09_XXX** parser. Reads a Gaussian 09 output file and returns a cell array of lines, with line endings stripped. Transparently handles CRLF line endings (as produced by Windows G09W builds) and ISO-8859-1 (Latin-1) encoding.

G16 parsers read files directly with the platform default encoding rather than calling a separate **G16_read_lines** helper; the 'Lines' parameter interface is identical across both toolboxes.

3 Core data extraction

3.1 G09_route / G16_route

```
route = G09_route(filename)
route = G16_route(filename)
route = G16_route(filename, 'Lines', lines)
```

Extracts the complete route section (the # line(s) between the two ---- separators) as a single string.

Option	Default	Description
'Lines'	{}	Pre-read file lines (Section 2.1); read normally if omitted

```
r = G09_route('indaco.log');
% '# opt freq=raman b3lyp/6-311++g(d,p) nosymm geom=connectivity field
=x+50'
```

Both `G09_route/G16_route` and `G09_charge_mult/ G16_charge_mult` now accept the 'Lines' option, consistent with the rest of the toolbox.

3.2 G09_structure / G16_structure

```
mol = G09_structure(filename)
mol = G09_structure(filename, 'step', 'last')
mol = G09_structure(filename, 'step', 'first')
mol = G09_structure(filename, 'step', N)

mol = G16_structure(filename)
mol = G16_structure(filename, 'orientation', 'standard')
mol = G16_structure(filename, 'orientation', 'input')
```

Extracts the molecular geometry from a Gaussian output file.

Difference: Gaussian 09 writes only "Input orientation:" blocks (never "Standard orientation:"), so `G09_structure` has no 'orientation' option. Gaussian 16 can write either, so `G16_structure` exposes 'orientation' = 'standard' (default, falls back to input if absent) | 'input' | 'auto'.

Option	Default	Description
'step'	'last'	Which geometry block to extract: 'first', 'last', or an integer index
'orientation'	'auto' (G16 only)	'standard' 'input' 'auto'
'Lines'	{}	Pre-read file lines; read normally if omitted

Field	Type	Description
.symbols	cell	Atomic symbols
.xyz	double	Coordinates in Å (X Y Z)
.Z	int	Atomic numbers
.Natoms	int	Number of atoms
.step	int	Index of the extracted step (1-based)
.n_steps	int	Total geometry blocks found in the file
.orientation	char	'Input orientation' (always, G09) or 'standard'/'input' (G16)
.filename	char	Source file name

```
mol = G09_structure('indaco.log');
mol = G16_structure('calc.out', 'orientation', 'input');
```

3.3 G09_energy / G16_energy

```
en = G09_energy(filename)
en = G09_energy(filename, 'step', 'last')
en = G09_energy(filename, 'step', N)
```

Extracts SCF energy and thermochemistry data.

Field	Type	Description
.SCF	double	SCF energy (Hartree)
.method	char	Method string, e.g. 'RB3LYP'
.ZPE_corr	double	Zero-point correction (Hartree)
.U_corr	double	Thermal correction to energy (Hartree)
.H_corr	double	Thermal correction to enthalpy (Hartree)
.G_corr	double	Thermal correction to Gibbs free energy (Hartree)
.E0	double	SCF + ZPE (Hartree)
.U	double	SCF + U_corr (Hartree, G16 only)
.H	double	SCF + H_corr (Hartree)
.G	double	SCF + G_corr (Hartree)
.S_JmolK	double	Entropy S (J mol ⁻¹ K ⁻¹ , G09)
.ZPE_kJ	double	Zero-point energy (kJ/mol)
.T, .P	double	Temperature (K), pressure (atm)
.has_thermo	logical	false for opt-only jobs (no freq); *_corr, E0, U, H, G, T, P are NaN in that case
.G_kJ, .H_kJ	double	G , H in kJ/mol (G09)
.filename	char	

Option	Default	Description
'step'	'last'	Which SCF block to extract: 'last', 'first', or integer N
'Lines'	{}	Pre-read file lines

3.4 G09_charge_mult / G16_charge_mult

```
[charge, mult] = G09_charge_mult(filename)
[charge, mult] = G16_charge_mult(filename)
[charge, mult] = G16_charge_mult(filename, 'Lines', lines)
```

Extracts molecular charge and spin multiplicity from the "Charge = ... Multiplicity = ..." line. Format identical across G09 and G16.

```
[q, m] = G09_charge_mult('indaco.log'); % q = 0, m = 1
```

3.5 G09_convergence / G16_convergence

```
cv = G09_convergence(filename)
cv = G09_convergence(filename, 'plot', true)

cv = G16_convergence(filename)
cv = G16_convergence(filename, 'plot', true)
```

Extracts geometry-optimisation convergence criteria (Max/RMS Force, Max/RMS Displacement) at every optimisation step, together with their thresholds and whether/when the job converged. Block format identical across G09 and G16.

Option	Default	Description
'plot'	false	Plot the four convergence series against their thresholds

Field	Type	Description
.MaxForce, .RMSForce	vector	Max / RMS force per step (a.u.)
.MaxDisp, .RMSDisp	vector	Max / RMS displacement per step (a.u.)
.thr_MaxForce, .thr_RMSForce, .thr_MaxDisp, .thr_RMSDisp	double	Corresponding convergence thresholds
.converged	logical	Whether all four criteria were satisfied
.conv_step	int/NaN	Step at which convergence was reached
.Nsteps	int	Number of optimisation steps found
.filename	char	

```
cv = G16_convergence('V_E00t.out', 'plot', true);
cv.converged
cv.conv_step
```

3.6 G09_dipole_polar / G16_dipole_polar

```
dp = G09_dipole_polar(filename)
dp = G09_dipole_polar(filename, 'units', 'Debye')

dp = G16_dipole_polar(filename)
dp = G16_dipole_polar(filename, 'units', 'SI')
```

Extracts the dipole moment and polarisability tensor.

Differences between G09 and G16: G09 prints "Exact polarizability:" as six values on a single line (upper triangle, row order), rather than G16's formatted Alpha(0;0)/Alpha(-w;w) blocks. G09 has no dynamic (laser-frequency-dependent) polarisability unless requested via Polar=OptRot or similar keywords, and additionally prints a "Diagonal vibrational polarizability:" line (3 values, returned in .alpha_vib). An approximate CPHF tensor, if printed by G09 ("Approx polarizability:"), is returned in .alpha_approx. G16 instead reports full dynamic polarisability entries per laser frequency (.alpha_dyn) and modal derivatives .dmu_dQ/.dalpha_dQ (related to IR intensities and Raman activities; for exact scaled values use G16_spectra instead).

Option	Default	Description
'units'	'au'	'au' 'Debye' (dipole only, G09) / 'Debye' (full, G16) 'SI'
'Lines'	{}	Pre-read file lines

Field	Description
.mu_x, .mu_y, .mu_z, .mu_tot, .mu_units	Dipole components, magnitude, unit string
.alpha_iso, .alpha_aniso	Isotropic polarisability, anisotropy
.alpha_tensor, .alpha_units	3×3 static tensor (a.u.), unit string
.alpha_vib, .has_alpha_vib	Diagonal vibrational polarisability (a.u.), logical (G09)
.alpha_approx	CPHF approximate tensor (a.u.), NaN if absent (G09)
.alpha_dyn, .N_dyn	Struct array of dynamic polarisability entries (per laser frequency), count (G16)
.dmu_dQ, .dalpha_dQ, .has_deriv	Dipole/polarisability derivatives w.r.t. normal modes, logical (G16)
.filename	

3.7 G09_nmodes / G16_nmodes

```
nm = G09_nmodes(filename)
nm = G09_nmodes(filename, 'modes', [5 10 20])

nm = G16_nmodes(filename)
```



```
nm = G16_nmodes(filename, 'section', 'last')
```

Extracts normal-mode displacement vectors.

G09 writes only one *Harmonic frequencies* section per file (even for opt+freq jobs), so `G09_nmodes` has no `'section'` option — it is implicit. G16 may contain more than one such section (e.g. a preliminary and a final one) and defaults to the last.

Option	Default	Description
'modes'	[]	Restrict output to specific mode indices
'section'	'last' (G16 only)	'last' 'first'
'Lines'	{}	Pre-read file lines

Field	Description
.freq,	.IR, Frequencies (cm^{-1}), IR intensities (KM/Mole), Raman activities
.Raman	($\text{\AA}^4/\text{AMU}$, [] if absent)
.redmass,	Reduced masses (AMU), force constants ($\text{mDyne}/\text{\AA}$)
.frconst	
.symmetry	Symmetry labels
.disp	$[N_{\text{atoms}} \times 3 \times N_{\text{modes}}]$ Cartesian displacement vectors
.Nmodes,	
.Natoms,	
.has_Raman,	
.filename	

3.8 G09_spectra / G16_spectra

```
sp = G09_spectra(filename)
sp = G09_spectra(filename, Name, Value, ...)

sp = G16_spectra(filename)
sp = G16_spectra(filename, 'section', 'last')
```

Extracts IR/Raman line spectra and generates Lorentzian-broadened continuous spectra.

G09 writes only one Harmonic-frequencies section per file, so `G09_spectra` needs no `'section'` parameter; `G16_spectra` exposes it to select which section to use when more than one is present.

Option	Default	Description
'FWHM'	10	Lorentzian FWHM (cm^{-1})
'xmin'	0	Lower wavenumber limit (cm^{-1})
'xmax'	4000	Upper wavenumber limit (cm^{-1})
'dx'	1	Grid step (cm^{-1})
'normalize'	false	Normalise continua to maximum = 1
'plot'	false	Generate figure
'section'	'last' (G16 only)	'last' 'first'
'Lines'	{}	Pre-read file lines

Field	Description
.freq, .Raman	.IR, Frequencies (cm^{-1}), IR intensities, Raman activities ([] if absent)
.has_Raman, .Nmodes	Logical, number of modes
.x, .Raman_cont	.IR_cont, Wavenumber grid, broadened IR/Raman continua
.FWHM, .filename	

```
sp = G16_spectra('calc.out', 'FWHM', 15, 'xmin', 400, 'plot', true);
```

4 Atomic charges

4.1 G09_charges / G16_charges

```
ch = G09_charges(filename)
ch = G09_charges(filename, 'type', 'APT')
ch = G09_charges(filename, 'mode', 'heavy')
ch = G09_charges(filename, 'plot', true, 'threshold', 0.05)
```

Extracts Mulliken or APT atomic charges and optionally renders them on the 3D structure, with an optional dipole-moment arrow overlay.

Robustness: handles all known Mulliken label variants across Gaussian revisions/OSes (e.g. "Mulliken atomic charges:" on G09 Rev. A.02/Windows and some Linux builds; "Mulliken charges:" on G09 Rev. B.01/C.01/D.01 and on all G16 revisions), by matching any line containing the charge-type keyword and "charges:", excluding "Sum of" and H-summed lines. The exact label found is reported in `ch.label`.

Option	Default	Description
'type'	'Mulliken'	'Mulliken' 'APT'
'mode'	'atom'	'atom' 'heavy' (H charges summed onto heavy atoms)
'plot'	true	Render labels on the 3D structure
'AtomScale'	0.35	CPK sphere scale
'BondTol'	1.30	Bond detection tolerance
'FontSize'	8	Charge-label font size
'ColorScale'	'RdBu'	'RdBu' (blue=neg/red=pos) 'none'
'threshold'	0	Hide labels with $ q < \text{threshold}$
'ShowDipole'	false	Overlay the total dipole moment vector
'DipoleOrigin'	'negcharge'	Arrow anchor: 'negcharge' (centroid of negative charges) 'poscharge' 'centroid' [x y z]
'DipoleScale'	1.0	Arrow length, Å per Debye
'DipoleColor'	[0 0.6 0]	Arrow colour
'DipoleLineWidth'	2.5	Arrow line width
'ShowDipoleLabel'	true	Annotate the arrow with $ \mu $
'DipoleFontSize'	11	Dipole label font size
'DipoleUnits'	'Debye'	Units for $ \mu $: 'Debye' 'au'
'Lines'	{}	Pre-read file lines

Field	Description
.symbols, .charges	Atomic symbols, per-atom charges (e)
.charges_H	H-summed charges on heavy atoms
.sum_q	Sum of all charges (≈ 0 for neutral molecules)
.type, .label	'Mulliken'/'APT', exact header label found
.Natoms	

4.2 G09_charges_fchk (G09 only)

```
ch_out = G09_charges_fchk(mol, ch)
ch_out = G09_charges_fchk(mol, ch, Name, Value, ...)
```

Visualises atomic charges directly from a `G09_fchk_read` struct, without requiring the corresponding `.log/.out` file. Mirrors the interface of `G09_charges` exactly, so the two can be used interchangeably once `data.mol` and `data.ch` are available.

Option	Default	Description
'mode'	'atom'	'atom' 'heavy' (requires <code>ch.charges_H</code> non-empty — <code>.fchk</code> files have no native H-summed data; use <code>G09_charges</code> on the <code>.log</code> for that)
'plot'	true	
'AtomScale'	0.35	
'BondTol'	1.30	
'FontSize'	8	
'ColorScale'	'RdBu'	'RdBu' 'none'
'threshold'	0	

```
data = G09_fchk_read('3typ.fchk');
ch    = G09_charges_fchk(data.mol, data.ch); %
      default
ch    = G09_charges_fchk(data.mol, data.ch, 'plot', false, 'threshold',
      0.05);
ch    = G09_charges_fchk(data.mol, data.ch, 'ColorScale', 'none'); %
      drop-in for G09_charges
```

Requires `mol` with fields `.symbols`, `.xyz`, `.Natoms`, `.filename`, and `ch` with fields `.charges`, `.symbols`, `.type`, `.Natoms` (`.charges_H` optional).

No equivalent `G16_charges_fchk` has been provided yet: since `.fchk` format is identical for G09 and G16, `G09_charges_fchk` can be used with `.fchk` files from either program version.

5 Formatted checkpoint (.fchk) support

5.1 G09_fchk_read (G09 only)

```
data = G09_fchk_read(filename)
data = G09_fchk_read(filename, 'verbose', false)
```

Reads a Gaussian 09/16 formatted checkpoint file (.fchk), generated with `formchk jobname.chk jobname.fchk`. Uses a single generic section parser (no hard-coded line counts) and is compatible with both G09 and G16 .fchk files.

Supersedes the old .fck/newzmat-format reader: reads the modern .fchk ASCII format, returns SI-ready fields (XYZ in Å, dipole in Debye), and needs no external helper functions. Despite the G09_ prefix, this function works on .fchk files produced by *either* G09 or G16, since the formatted-checkpoint layout does not depend on the Gaussian version.

Option	Default	Description
'verbose'	true	Print progress messages while parsing

Output struct data (grouped by section):

Field	Description
.title, .method, .basis	First line, method (e.g. 'RB3LYP'), basis set (e.g. 'def2SVPP')
.Nat, .charge, .mult	Number of atoms, molecular charge, spin multiplicity
.Nelec, .Nalpha, .Nbeta	Total, alpha, beta electron counts
.Nbasis, .Nbasis_indep	Basis functions, independent basis functions
.symbols, .AN, .masses	Atomic symbols, atomic numbers, real atomic masses (AMU)
.xyz, .xyz_bohr	Coordinates (Å), coordinates (Bohr)
.SCF_energy, .total_energy, .virial_ratio	SCF energy, total energy (Hartree), virial ratio (≈ 2)
.alpha_orb_energies, .beta_orb_energies	MO energies (Hartree); beta is [] if RHF
.alpha_MO_coeff	Alpha MO coefficients (column-major)
.mulliken_charges	Mulliken charges (e)
.HOMO_idx, .HOMO_eV, .LUMO_eV, .gap_eV	Frontier-orbital summary
.gradient, .rms_force	Cartesian gradient (Hartree/Bohr), RMS force

6 Molecule and normal-mode visualisation

6.1 G09_draw_molecule / G16_draw_molecule

```
G09_draw_molecule(mol)
G09_draw_molecule(mol, 'Name', Value, ...)
```

Renders a 3D CPK ball-and-stick model from a mol struct (as returned by G09_structure/G16_structure), with bonds detected automatically from a covalent-radius criterion.

Option	Default	Description
'AtomScale'	0.35	Sphere radius as a fraction of the covalent radius
'BondTol'	1.30	Bond tolerance: bonded if $d < (r_1 + r_2) \times \text{BondTol}$
'ShowLabels'	true	Show atom labels ("C1", "H2", ...)
'ShowLegend'	true	Show the element-colour legend
'Title'	filename	Figure title
'BgColor'	[0.95 0.95 0.95]	Axes background colour
'Ax'	(new figure)	Draw into an existing axes handle
'ShowAxes'	false	Draw a small Cartesian X/Y/Z reference triad, anchored at the lower-left-front corner of the plotted molecule
'AxesLength'	[]	Length of the X/Y/Z arrows, in Å ([] = auto, 20% of the molecule's bounding-box diagonal)

```
mol = G09_structure('zeatin.out');
G09_draw_molecule(mol);
G09_draw_molecule(mol, 'ShowLabels', false, 'AtomScale', 0.4);
G09_draw_molecule(mol, 'ShowAxes', true);
```

No differences between the G09 and G16 versions beyond the input struct's origin.

6.2 G09_draw_mode / G16_draw_mode

```
G09_draw_mode(mol, nm, mode_idx)
G09_draw_mode(mol, nm, mode_idx, 'Name', Value, ...)
```

Renders a single vibrational normal mode as displacement arrows over the CPK structure.

Option	Default	Description
'Scale'	1.5	Arrow length scale
'ArrowColor'	[1 0.4 0.1]	Arrow colour
'AtomScale'	0.35	CPK sphere scale factor
'BondTol'	1.30	Bond detection tolerance
'ShowLabels'	false	Show atom index labels
'FlipSign'	false	Invert all arrow directions

Why 'FlipSign' exists. Normal-mode eigenvectors have an arbitrary overall sign — a mode and its 180°-phase-shifted twin are physically identical. If the arrows point opposite to GaussView's rendering for a given mode, set 'FlipSign', true to flip them. This is purely cosmetic and has no effect on frequencies or intensities.

6.3 G09_modeViewer / G16_modeViewer

```
G09_modeViewer(filename)
G09_modeViewer(filename, 'Name', Value, ...)
```

Interactive selector GUI for browsing vibrational modes. Reads the structure and all normal modes via `G09_structure/G09_nmodes`, then opens a window listing every mode (index, frequency, symmetry). Selecting an entry calls `G09_draw_mode` to render that mode in a new figure window; previously drawn mode figures stay open, so different modes (or render settings) can be compared side by side.

Window controls:

- *Order by:* sort the mode list by mode number (default), IR intensity, or Raman intensity (descending; only offered when Raman data is present). The current selection is preserved across reordering.
- A text box to set a custom title on the current mode figure.
- A control for every `G09_draw_mode` option (`Scale`, `ArrowColor`, `AtomScale`, `BondTol`, `ShowLabels`, `FlipSign`); changing any of them immediately redraws the selected mode in a new figure. Arrow colour can be set from a named preset (Orange/Red/Blue/Green/Black/ Magenta) or a custom colour.
- A *Save figure...* button exporting the target mode figure to JPEG, EPS, or PDF (EPS/PDF as true vector graphics).

Name-Value arguments passed to `G09_modeViewer` pre-populate the corresponding option control(s), e.g. `G09_modeViewer('file.log', 'Scale', 2, 'ShowLabels', true)`.

```
G09_modeViewer('V_E00t.out');
G16_modeViewer('INDACO_E025_ppDP.LOG', 'Scale', 2, 'ShowLabels', true)
;
```

See also `G09_structure/G16_structure`, `G09_nmodes/G16_nmodes`, `G09_draw_mode/G16_draw_mode`.

7 Orbital energy analysis

7.1 G09_orbital_energies / G16_orbital_energies

```
oe = G09_orbital_energies(filename)
oe = G09_orbital_energies(filename, 'step', 'last')
oe = G09_orbital_energies(filename, 'step', N)
```

Extracts molecular orbital energies (HOMO/LUMO and the full occupied/virtual spectrum) from a Gaussian 09/16 output file, by parsing the "Alpha occ. eigenvalues --" / "Alpha virt. eigenvalues --" lines printed by Gaussian's population analysis (and the corresponding "Beta" lines for open-shell calculations). These blocks repeat once per SCF calculation in the file (e.g. once per optimisation step); the 'step' option selects which block to report, consistently with G09_energy/G09_structure.

Option	Default	Description
'step'	'last'	Which eigenvalue block to use: 'first', 'last', or an integer index

Output struct oe (energies in Hartree unless noted):

Field	Type	Description
.alpha_occ	vector	Occupied alpha orbital energies
.alpha_virt	vector	Virtual alpha orbital energies
.beta_occ	vector	Occupied beta orbital energies ([] if closed-shell)
.beta_virt	vector	Virtual beta orbital energies ([] if closed-shell)
.has_beta	logical	true for open-shell (UHF/UKS) calculations
.HOMO / .LUMO	double	Highest occupied / lowest unoccupied orbital energy
.gap	double	LUMO -- HOMO
.HOMO_alpha / .LUMO_alpha	double	Alpha HOMO/LUMO (== .HOMO/.LUMO for closed-shell)
.HOMO_beta / .LUMO_beta	double	Beta HOMO/LUMO ([] if closed-shell)
.HOMO_eV / .LUMO_eV / .gap_eV	double	Same three quantities, in eV
.step	int	Block index actually used
.Nsteps	int	Number of eigenvalue blocks found in the file
.filename	char	Source file name

```
oe = G09_orbital_energies('V_E00t.out');
fprintf('HOMO-LUMO gap = %.3f eV\n', oe.gap_eV);
```

For open-shell (UHF/UKS) calculations, HOMO and LUMO are computed as $\max(\text{HOMO}_\alpha, \text{HOMO}_\beta)$ and $\min(\text{LUMO}_\alpha, \text{LUMO}_\beta)$ respectively, so .gap always refers to the smallest alpha/beta HOMO-LUMO gap. No differences between the G09 and G16 versions beyond the parsed file's origin.

7.2 G09_draw_orbital / G16_draw_orbital

```
G09_draw_orbital(oe)
G09_draw_orbital(oe, 'Name', Value, ...)
```

Draws a molecular-orbital energy-level diagram from the struct returned by `G09_orbital_energies/G16_orbital_energies` showing a user-selected number of occupied/virtual levels around the frontier orbitals and highlighting the HOMO–LUMO transition with an arrow labelled with the gap value.

Option	Default	Description
'NLevels'	5	Number of occupied/virtual levels shown on each side of the frontier orbitals
'Units'	'eV'	Energy units: 'eV' or 'Hartree'
'Ax'	(new figure)	Draw into an existing axes handle
'OccColor'	[0.25 0.25 0.70]	Colour for non-frontier occupied levels
'VirtColor'	[0.70 0.30 0.25]	Colour for non-frontier virtual levels
'HOMOColor'	[0.00 0.45 0.85]	Highlight colour for the HOMO
'LUMOCOLOR'	[0.90 0.35 0.00]	Highlight colour for the LUMO
'ArrowColor'	[0.15 0.60 0.15]	HOMO→LUMO transition arrow colour
'ShowLabels'	true	Annotate each level with its energy value
'ShowSpins'	true	Draw a paired-electron arrow glyph ($\uparrow\downarrow$) on occupied levels
'FontSize'	8	Font size for level energy-value labels
'FrontierFontSize'	13	Font size for the HOMO/LUMO tags and the gap annotation
'ArrowHeadSize'	0.2	HOMO–LUMO arrow head size, as a fraction of the arrow length (passed to <code>quiver</code>)
'Title'	filename or 'Orbital Energy Diagram'	Figure title

```
oe = G09_orbital_energies('a1.out');
G09_draw_orbital(oe)
G09_draw_orbital(oe, 'NLevels', 8, 'Units', 'Hartree')
```

Requires the `oe` struct returned by `G09_orbital_energies/ G16_orbital_energies` and errors if `alpha_occ/alpha_virt` are missing or empty. No differences between the G09 and G16 versions beyond the input struct's origin.

8 Response properties (G16 only)

8.1 G16_hyperpolar

```
hp = G16_hyperpolar(filename)
hp = G16_hyperpolar(filename, 'units', 'au')      % default
hp = G16_hyperpolar(filename, 'units', 'esu')    % 10-30 esu
hp = G16_hyperpolar(filename, 'units', 'SI')     % 10-50 C3 m3 J-2
```

Extracts the dipole hyperpolarisability β (static, dynamic, and vibrational contributions) from a Gaussian 16 output file.

Option	Default	Description
'units'	'au'	'au' 'esu' (10 ⁻³⁰ esu) 'SI' (10 ⁻⁵⁰ C ³ m ³ J ⁻²)

Field	Description
.beta0.par_z,	Parallel / perpendicular component of static $\beta(0;0,0)$ along z
.beta0.perp_z	
.beta0.vec_x/y/z,	Vector components and magnitude $ \beta_{\text{vec}} $
.beta0.beta_vec	
.beta0.tensor	All Cartesian tensor components ($xxx, xyx, \dots, zzz$)
.beta_dyn	Struct array, one entry per laser frequency: .lambda_nm, .par_z, .perp_z, .vec_x/y/z, .beta_vec, .tensor
.N_dyn	Number of dynamic (laser) frequencies found
.beta_vib,	Diagonal vibrational hyperpolarisability, logical
.has_vib	
.filename	

```
hp = G16_hyperpolar('V_E00t.out');
hp.beta0.beta_vec          % static beta modulus, a.u.
hp.beta_dyn(1).lambda_nm  % wavelength of the first dynamic entry (nm)
hp.beta_dyn(1).beta_vec   % |beta| at that frequency
```

8.2 G16_tddft

```
td = G16_tddft(filename)
td = G16_tddft(filename, 'nstates', N)      % first N states only
td = G16_tddft(filename, 'plot', true)     % also generate UV-Vis plot
td = G16_tddft(filename, 'FWHM_eV', 0.3)   % broadening FWHM (eV)
```

Extracts TD-DFT excited states and generates a Gaussian-broadened UV-Vis spectrum.

Option	Default	Description
'nstates'	Inf	Load only the first N excited states
'plot'	false	Generate the UV-Vis plot
'FWHM_eV'	0.30	Gaussian broadening FWHM (eV)

Field	Description
.n, .mult	State index; multiplicity label (e.g. 'Singlet-A', 'Triplet-B')
.eV, .nm, .f	Excitation energy (eV), wavelength (nm), oscillator strength
.S2, .has_S2	$\langle S^2 \rangle$ expectation value (0 for pure singlets), logical
.trans	Cell array, one $[N_{\text{contrib}} \times 3]$ matrix per state: columns [MO_from, MO_to, coeff] (MO_to < 0 denotes a de-excitation)
.Nstates	Total number of excited states loaded
.x_nm, .eps_cont	Wavelength grid (100–1000 nm), Gaussian-broadened UV-Vis spectrum (arb. units)
.FWHM_eV, .filename	

```

td = G16_tddft('violacein_td.out');
plot(td.x_nm, td.eps_cont)      % UV-Vis spectrum
stem(td.nm, td.f)              % transition stem plot
td.trans{2}                    % transitions contributing to the 2nd
    state

```

No G09 equivalent is provided: TD-DFT excited-state analysis and hyperpolarisability response properties in this toolbox are currently supported only for Gaussian 16 output files.

9 Structural analysis utilities

9.1 G09_get_bond_length / G16_get_bond_length

```
bondTable = G09_get_bond_length(mol)
bondTable = G09_get_bond_length(mol, 'Name', Value, ...)
```

Builds the bond-length table of a molecule from a `mol` struct (fields `.symbols`, `.xyz`), detecting bonds via a covalent-radius criterion, and returns a MATLAB table.

Option	Default	Description
'Tolerance'	1.15	Multiplicative factor on the sum of covalent radii for the bonding criterion
'IncludeH'	true	Include bonds involving hydrogen atoms
'SortBy'	'distance'	Sort by 'distance' (ascending) or 'atom' (atom index)
'SaveAs'	''	Path (.xlsx or .csv) to save the table to; not saved if omitted

Output columns: Atom1, Sym1, Atom2, Sym2, Distance Ang.

```
T = G09_get_bond_length(mol, 'SortBy', 'atom');
T = G16_get_bond_length(mol, 'IncludeH', false, 'SaveAs', 'bonds.xlsx');
```

The bonding criterion is purely geometric (covalent radii), not derived from Gaussian's own connectivity/topology matrix — suitable for quick structural analysis, not as the "official" G09/G16 connectivity.

9.2 G09_gaussian_version / G16_gaussian_version

```
gv = G09_gaussian_version(filename)
gv = G16_gaussian_version(filename)
```

Detects which Gaussian version/revision produced a `.out/.log/.fchk` file.

For `.out/.log` files, reads the "Gaussian NN, Revision X.YY," citation line printed near the top of the file (works for any NN: 09, 16, ...). `.fchk` files do not store this information at all; in that case the function looks for a sibling `.log/.out` file with the same base name in the same folder and reads the version from there. If no sibling file is found, `.major/.revision/.full` are returned empty and a warning is issued.

Field	Description
<code>.major</code>	Gaussian major version, e.g. 9, 16 ([]) if unknown)
<code>.revision</code>	Revision string, e.g. 'A.02', 'C.01' ('' if unknown)
<code>.full</code>	Citation line as printed, e.g. 'Gaussian 09, Revision A.02'
<code>.source</code>	'out/log' 'fchk-sibling:<path>' 'unknown'
<code>.filename</code>	

```
gv = G09_gaussian_version('a1.out');          % gv.major = 16, gv.
revision = 'C.01'
```

```
gv = G16_gaussian_version('molecule.fchk'); % falls back to molecule.
out/.log if present
```

9.3 G09_restart (G09 only)

```
gjf_file = G09_restart(filename)
gjf_file = G09_restart(filename, 'Name', Value, ...)
```

Generates a Gaussian 09/16 input file (.gjf) to restart a calculation from a geometry found in an existing .log/.out file.

Option	Default	Description
'step'	'last'	'last' 'first' integer N — which geometry step to restart from
'output'	<base>_restarted.gjf	Output .gjf path
'extra_route'	''	Additional keywords appended to the route section
'nproc'	0	%nprocshared override (0 = keep/omit)
'mem'	''	%mem override ('' = keep/omit)

Gaussian 09 Windows output files often lack a %chk line; in that case the checkpoint name is derived from the source file name. No G16_restart has been provided yet; the generated .gjf is valid for restarting under either Gaussian 09 or Gaussian 16.

Returns the path of the generated .gjf file.

9.4 G09_list / G16_list

```
G09_list()
T = G09_list()

G16_list()
T = G16_list()
```

Lists every G09_*.m/G16_*.m function found in the toolbox folder, printing the function name and its H1 description line (sorted alphabetically). Also returns the same information as a table with columns .Name, .Description, .File.

```
G09_list();
T = G16_list();
T(contains(T.Description, 'dipole', 'IgnoreCase', true), :)
```

10 One-call data extraction

10.1 G09_read_all / G16_read_all

```
T = G09_read_all(filename)
T = G16_read_all(filename)
```

Runs the full set of G09_XXX/G16_XXX parsers on a single output file and collects the results into one struct T. The file is read from disk once and the resulting lines are passed to every sub-parser via their 'Lines' option, instead of each sub-parser independently re-reading and re-splitting the file.

Field	Source function	Description
.charge	G09/G16_charges	Mulliken and APT atomic charges (plotting disabled)
.energy	G09/G16_energy	SCF energy and thermochemistry
.structure	G09/G16_structure	Optimized molecular structure
.dipolar	G09/G16_dipole_polarizability	Dipole moment and polarisability
.nmodes	G09/G16_nmodes	IR/Raman vibrational normal modes, if present
.spectra	G09/G16_spectra	Frequencies, IR/Raman intensities, simulated spectra
.route	G16_route (G16 only)	Gaussian route section details
.chargemol.charge	G16_charge_mult	Total molecular charge, spin multiplicity
.chargemol.mol	(G16 only)	

```
T = G16_read_all('zeatin.out');
disp(T.energy)
G16_draw_molecule(T.structure);
```

Known discrepancy: the current G09_read_all does *not* populate .route or .chargemol, unlike G16_read_all, which retains both. If parity with G16_read_all is required, add

```
T.route = G09_route(filename, 'Lines', lines);
[c, m] = G09_charge_mult(filename, 'Lines', lines);
T.chargemol.charge = c;
T.chargemol.mol = m;
```

to G09_read_all.m.

11 Python package: G16parser

G16parser is a Python 3 port of the G16 MATLAB toolbox, covering the same parsing, checkpoint-analysis, and static-visualisation functionality described in the previous sections. It is installable as a standard editable package and exposes one `g16_xxx` function per MATLAB G16_xxx counterpart, each returning a `Struct` object (attribute access, e.g. `mol.xyz`, `ch.dipole.Debye`) instead of a MATLAB struct.

```
cd G16parser
pip install -e .
```

```
import G16parser as g16

mol = g16.g16_structure('molecule.out')
g16.g16_draw_molecule(mol, show_axes=True)

ch = g16.g16_charges('molecule.out', show_dipole=True)

oe = g16.g16_orbital_energies('molecule.out')
g16.g16_draw_orbital(oe)

T = g16.g16_read_all('molecule.out')    # everything in one call,
    single file read
```

Requirements: Python ≥ 3.9 , numpy, pandas, matplotlib (installed automatically via `pip install -e .`). No Gaussian installation or compiled dependency is needed.

11.1 Function reference

Every function mirrors its MATLAB G16_xxx counterpart in name, parameters (in `snake_case`), defaults, and returned fields — see the corresponding MATLAB subsection earlier in this manual for full parameter/field documentation. All extraction functions accept an optional `lines=` parameter (pre-read file lines) mirroring the MATLAB 'Lines' option; `g16_read_all` uses this internally to read the file exactly once.

Function	Description
<code>g16.structure</code>	Molecular geometry (symbols, xyz as <code>np.ndarray</code> , atom count)
<code>g16.energy</code>	SCF energy and thermochemistry
<code>g16.convergence</code>	Geometry-optimisation convergence criteria per step (<code>plot=True</code> for a matplotlib figure)
<code>g16.charges</code>	Mulliken/APT atomic charges, optional dipole overlay and 3D plot
<code>g16.dipole_polar</code>	Dipole moment and (static/dynamic) polarisability
<code>g16.nmodes</code>	Vibrational normal-mode displacement vectors
<code>g16.spectra</code>	IR/Raman spectra (stick + Lorentzian-broadened continuum)
<code>g16.orbital_energies</code>	HOMO/LUMO and the full occupied/virtual MO spectrum
<code>g16.charge_mult</code>	Molecular charge and spin multiplicity
<code>g16.route</code>	Route section string
<code>g16.get_bond_length</code>	Bond-length table (pandas DataFrame) from covalent radii
<code>g16.gaussian_version</code>	Detects the Gaussian version/revision (works on <code>.fchk</code> via a sibling <code>.log/.out</code>)
<code>g16.hyperpolar</code>	Dipole hyperpolarisability (β)
<code>g16.tddft</code>	TD-DFT excited states
<code>g16.read_all</code>	Runs the full extraction set in one call, reading the file only once
<code>g16.draw_molecule</code>	3D CPK ball-and-stick render (matplotlib <code>mplot3d</code>)
<code>g16.draw_mode</code>	3D structure with a vibrational mode's displacement arrows
<code>g16.draw_orbital</code>	Orbital energy-level diagram with HOMO–LUMO gap arrow
<code>g16.mode_viewer</code>	Interactive Tkinter window to browse/render vibrational modes, sortable by mode number, IR, or Raman intensity
<code>g16.list</code>	Lists every function in the toolbox with its one-line description (pandas DataFrame)

11.2 Selected signatures

```

g16.structure(filename, orientation="auto", step="last", lines=None)
g16.energy(filename, step="last", lines=None)
g16.convergence(filename, plot=False, lines=None)
g16.nmodes(filename, section="last", modes=None, lines=None)
g16.spectra(filename, FWHM=10, xmin=0, xmax=4000, dx=1, normalize=
    False,
        plot=False, section="last", lines=None)
g16.charges(filename, type="Mulliken", mode="atom", plot=True,
    atom_scale=0.35, bond_tol=1.30, font_size=8, color_scale="
        RdBu",
    threshold=0.0, show_dipole=False, dipole_origin="negcharge
        ",
    dipole_scale=1.0, dipole_color=(0, 0.6, 0),
    dipole_line_width=2.5,
    show_dipole_label=True, dipole_font_size=11, dipole_units
        ="Debye",
    lines=None)
g16.draw_molecule(mol, atom_scale=0.35, bond_tol=1.30, show_labels=
    True,

```



```

        show_legend=True, title="", bg_color=(0.95, 0.95,
        0.95),
        ax=None, show_axes=False, axes_length=None)
g16_mode_viewer(filename, scale=1.5, atom_scale=0.35, bond_tol=1.30,
        show_labels=False, flip_sign=False, arrow_color=None,
        font_size=10, **extra_opts)
g16_get_bond_length(mol, tolerance=1.15, include_h=True,
        sort_by="distance", save_as="")
g16_hyerpolar(filename, units="au", lines=None)
g16_tddft(filename, nstates=math.inf, plot=False, fwhm_ev=0.30, lines=
        None)

```

11.3 Notes on the port

3D interactivity. matplotlib’s `mplot3d` has no MATLAB-style `rotate3d` widget; use the interactive backend’s normal mouse/toolbar controls instead.

Backend selection. The package forces the TkAgg matplotlib backend (if Tk is available and pyplot has not been imported yet) instead of the native macOS backend: on some older matplotlib releases, interactively rotating a 3D plot with the native Cocoa backend can segfault the whole Python process. Call `matplotlib.use(...)` yourself *before* importing `G16parser` if a different backend is needed.

Bug found during porting, MATLAB-side only: in the original `G16_tddft.m`, MATLAB’s `regexp` silently drops the optional trailing `<S**2>=` capture group (a MATLAB-specific regex-engine quirk), so `td.S2` is always `NaN` in MATLAB even when the tag is present in the file. This Python port’s `re`-based parser does not have that bug and extracts `S2` correctly. Consider this a known discrepancy between the two implementations rather than a porting error.

`g16_mode_viewer` is a Tkinter rewrite of `G16_modeViewer.m`’s `uifigure` GUI, with the same controls (mode selector, order-by, per-option redraw, title, save-as PDF/EPS/JPEG). One simplification: “target figure” always means the most recently drawn mode window, not whichever window the user last clicked on (MATLAB’s `groot().CurrentFigure` has no simple Tkinter equivalent). In an IPython/Spyder console with the Tk graphics backend active, the viewer detects the running event loop and returns to the prompt immediately instead of blocking; run as a plain script, it blocks until the viewer window is closed.

Known issue: `g16_mode_viewer` can segfault when run from *inside* Spyder’s IPython console (a crash inside ipykernel’s own periodic Tk event-loop pump, not in this toolbox’s code) — confirmed working correctly when run from a plain terminal/script instead:

```
python3 -c "import G16parser as g16; g16.g16_mode_viewer('file.out
    ')"
```

All other functions, including static plots, work fine inside Spyder.

12 Publication and licensing

12.1 Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the author(s) used Claude (Anthropic) to assist with code drafting, refactoring, and documentation of the G09/G16 toolbox and its associated user manual. After using this tool, the author(s) reviewed and edited the content as needed and take full responsibility for the content of the publication.

12.2 Note on the Use of AI-Assisted Development Tools

Portions of the codebase and documentation described in this manual — including the G09_/G16_ MATLAB function pairs, the `g16toolbox` Python package, the `G09_modeViewer.m`/`G16_modeViewer.m` interactive GUIs, and this reference manual — were developed with the assistance of Claude (Anthropic), an AI-based coding tool, used interactively under the direct supervision of the author. All AI-generated code was tested against reference Gaussian 09/16 output files and reviewed by the author prior to inclusion. Algorithm design, scientific validation, and correctness of results remain the sole responsibility of the author.

12.3 Licence

This software is released under the **BSD 3-Clause License**, an OSI-approved open source license accepted by SoftwareX. See the `LICENSE.txt` file distributed with the source code for the full licence text.